

Empirical Evaluation of Agentic Architectures: Comparing Planner-Executor and Reflection Patterns Across Visual Reasoning and Constraint-Based Planning Tasks

Sardar Sardarov
Eötvös Loránd University (ELTE)

July 1, 2026

Abstract

Does agentic scaffolding provide generalized reasoning improvements for Large Language Models (LLMs), or is its efficacy strictly domain-dependent? This study investigates the impact of two multi-agent architectures—Planner-Executor (task decomposition) and Solver-Critic (iterative reflection)—across diametrically opposed domains: visual-spatial reasoning (ARC-AGI) and text-based constraint satisfaction (Meeting Scheduling). Evaluating a deterministic base model (**gpt-5.1**) across varying complexities, empirical results reveal no universally superior architecture. For rigid algorithmic scheduling, the Planner-Executor framework proved optimal, achieving 100% accuracy by externalizing mathematical logic to prevent cascading errors. Conversely, upfront planning collapsed in the abstract spatial domain. While the Solver-Critic loop provided the only viable recovery mechanism for visual topologies, it exhibited extreme token volatility and hit a hard “reflection ceiling,” failing on high-complexity tasks due to context dilution and attention collapse. Ultimately, agentic workflows face strict diminishing returns: deploying heavy reflection loops on semantic tasks causes severe architectural overkill, while applying sequential planners to dynamic spatial tasks results in unrecoverable geometric failures.

1 Introduction

While frontier models increasingly rely on internal reasoning loops, deploying standard base models efficiently requires “agentic scaffolding”—external multi-agent systems that enforce step-by-step logic [5, 6]. This study investigates whether the efficacy of specific scaffolding architectures is universally beneficial or intrinsically tied to a task’s domain and difficulty.

To rigorously test this, we utilize the Abstraction and Reasoning Corpus (ARC-AGI), a definitive measure of “fluid intelligence” that requires systems to infer hidden rules from a few examples and generalize them to novel spatial situations [1].

Fundamentally, ARC-AGI is a visual-spatial challenge where shapes scale, rotate, and interact based on abstract topological rules. Because we evaluate text-based Large Language Models (LLMs), these 2D grids are serialized into nested text arrays. This strips away visual intuition, forcing the model to rely purely on spatial arithmetic [2]—a challenge that frontier models have only recently overcome by wrapping themselves in advanced, highly expensive multi-agent reflection loops [3, 4].

To isolate the structural impact of these scaffolds, we evaluate a standard base model (**gpt-5.1**) across two diametrically opposed domains. We contrast ARC-AGI’s fluid topological challenges with a purely text-based constraint satisfaction problem: multi-agent, cross-timezone scheduling. This rigid algorithmic task demands strict parameter adherence and sequential logic. By comparing performance across these two domains, we can determine if a multi-agent framework provides a generalized reasoning enhancement, or if its efficacy is strictly bounded by how well the framework’s structure maps to the task’s cognitive requirements.



Figure 1: An example spatial reasoning task from the ARC-AGI dataset (Task #3aa6fb7a). The underlying transformation rule requires identifying the inner corner of each pink shape and injecting a new yellow pixel. While this topological relationship is immediately intuitive to human perception, base LLMs struggle to infer this geometry when the 2D matrix is flattened into a text-based array.

To formalize this investigation, we propose the following core research questions:

Research Questions

- How do structured multi-agent communication patterns—specifically Planner-Executor (Task Decomposition) and Solver-Critic (Iterative Reflection)—affect performance, token efficiency, and error rates across spatial reasoning versus text-based constraints?
- Does separating the planning phase from execution reliably mitigate logic errors in constraint satisfaction tasks compared to a single-agent baseline?
- Do bounded reflection loops provide sufficient corrective feedback to solve abstract spatial transformations, and do the performance gains mathematically justify the token overhead?

Initial Hypotheses

We establish the following hypotheses regarding structural efficacy and computational overhead:

- **H1 (Domain Superiority):** The Planner-Executor architecture will excel in Meeting Scheduling because text-based constraint sorting benefits heavily from sequential logic decomposition. Conversely, the Solver-Critic loop will perform better in ARC-AGI because abstract visual patterns are difficult to plan sequentially, making an iterative "try-and-verify" approach more effective.
- **H2 (Difficulty Scaling):** On Easy tasks across both domains, the Single-Agent Baseline will achieve comparable accuracy to multi-agent systems but at a fraction of the computational cost. The performance gap will only become statistically significant on Hard tasks.
- **H3 (Cost-Benefit Tradeoff):** While the Solver-Critic architecture will achieve the highest overall robustness, it will exhibit severe token consumption and latency overhead. The Planner-Executor will emerge as a more cost-efficient compromise for structured algorithmic pipelines.

2 Related Work

2.1 LLM-Based Solutions for ARC-AGI

Historically, applying Large Language Models to the Abstraction and Reasoning Corpus (ARC-AGI) has yielded poor results. Because LLMs process 2D grids as flattened text tokens, they

lack native topological awareness and operate primarily as superficial pattern matchers [2].

To circumvent this, researchers shifted toward Program Synthesis, prompting models to write transformation code rather than predicting grids directly. Greenblatt [3] achieved a landmark 50% accuracy by pairing GPT-4o with massive test-time compute, generating over 8,000 candidate Python programs per task. This brute-force methodology peaked with OpenAI’s o3 model, which utilized deep evolutionary refinement to reach 87.5% on ARC-AGI [4]. However, this milestone required exorbitant computational resources—generating up to 5.7 billion tokens per task—rendering it economically unviable for standard deployments.

2.2 Agentic and Multi-Agent Scaffolding

To achieve efficient problem-solving without massive compute overhead, researchers have increasingly relied on multi-agent scaffolding. A foundational framework is Reflexion [5], which utilizes an "Actor-Evaluator" loop to enable self-correction via linguistic feedback.

While highly effective in text-based benchmarks, applying reflection directly to spatial arrays often induces the "ping-pong effect," where models hallucinate geometric errors and iteratively degrade valid solutions. To address this, specialized scaffolds like Planner-Executor workflows have emerged. Wang et al. [6] demonstrated the efficacy of "Plan-and-Solve" prompting, which isolates the logical planning phase from execution to minimize cascading arithmetic errors, heavily influencing modern structured reasoning paradigms.

3 Methodology

3.1 Implementation Details

To orchestrate the multi-agent workflows, we utilized **LangGraph**, selected for its native support of cyclical routing and persistent memory states (**AgentState**). The **gpt-5.1** model served as the core inference engine. To ensure strict reproducibility and isolate the structural impact of the agents, the model was locked to a deterministic state (**temperature = 0.0**) without domain-specific fine-tuning; all behavioral conditioning was achieved exclusively through persona-based system prompts.

3.2 Architectures

We evaluated three specific system architectures:

- **Single LLM (Baseline):** A standard zero-shot prompt requesting immediate task execution.
- **Solver-Critic:** A cyclic state machine where a Critic audits the Solver’s output against domain constraints. Loops were strictly capped at 3 iterations to prevent infinite hallucination cycles.
- **Planner-Executor:** A sequential pipeline where a Planner generates a step-by-step algorithm executed blindly by the Executor node.

3.3 Task Domain and Difficulty Categorization

Tasks within both domains were segregated into "Easy" and "Hard" categories based on two dimensions of cognitive load: *Task Complexity* (concurrent variable tracking) and *Planning Horizon* (interdependence of sequential sub-tasks).

3.3.1 Visual-Spatial Reasoning (ARC-AGI)

Difficulty scales based on changes in grid dimensions, unique color count, and total shape count.

- **Easy Tasks:** Feature static dimensions and constant colors, keeping the fundamental geometry intact and requiring a minimal planning horizon.
- **Hard Tasks:** Exhibit simultaneous changes in size, color, and shape. This spikes *task complexity* by forcing the model to track spatial scaling and semantic assignments concurrently. It also extends the *planning horizon*, necessitating a strict multi-step sequence (e.g., calculating boundaries, computing new output dimensions, and mapping pixels) where a single geometric miscalculation causes cascading failures.

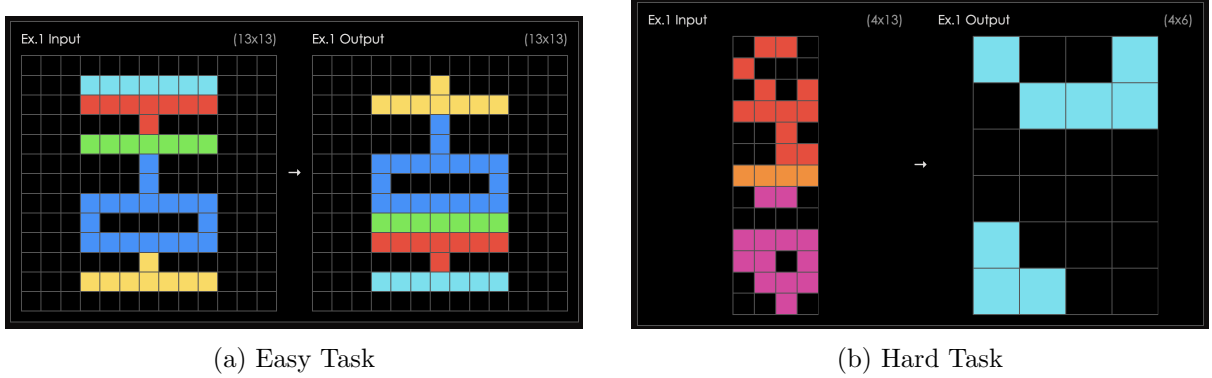


Figure 2: A visual comparison of task complexity. The Easy task (a) requires simple intra-grid movement, while the Hard task (b) forces the model to track dynamic shape counts and calculate a new output matrix size.

3.3.2 Constraint-Based Planning (Scheduling)

Difficulty scales based on the volume of temporal variables and timezone dependencies.

- **Easy Tasks:** Feature a flat planning horizon, balancing only two local variables with a single shared constraint, allowing for immediate intersection calculation.
- **Hard Tasks:** Scale *task complexity* via multi-variable expansion (3-4 participants), multi-dimensional rules (specific days), and global conditional constraints (lunch policies). This forces a deep *planning horizon*: the agent must sequentially isolate local windows, calculate precise UTC offsets, standardize the timeline, and compute the final intersection. A single arithmetic error in UTC conversion inevitably breaks the entire algorithmic chain.

4 Experiments

4.1 Experimental Setup and Dataset

The evaluation was conducted across a curated dataset of 16 total tasks, evenly divided between the two target domains: 8 Visual-Spatial tasks (ARC-AGI) and 8 Constraint-Based Planning tasks (Meeting Scheduling). Within each domain, the tasks were further divided into 4 "Easy" and 4 "Hard" variants, categorized according to the complexity metrics defined in the Methodology.

Although the `gpt-5.1` model was configured for a deterministic generation state (`temperature = 0.0`), minor variations in API responses can occasionally occur under high cognitive load. To account for this variance, the evaluation protocol allowed each architecture up to three independent attempts per task if it initially failed.

4.2 Evaluation Metrics

To assess the performance and efficiency tradeoffs of the multi-agent architectures, we captured three primary metrics across all runs:

- **Task Accuracy (Binary Pass/Fail):** A strict exact-match evaluation. For the scheduling domain, the output had to match the exact UTC time intersection. For the ARC-AGI domain, the predicted grid had to be a mathematically perfect 2D array match to the ground truth. Partial spatial matches were recorded as failures.
- **Token Overhead (Efficiency):** The total sum of input and output tokens consumed per task. This metric is critical for quantifying the financial and computational cost of multi-agent scaffolding compared to the baseline zero-shot approach.
- **Execution Latency (Time-to-Solution):** The total execution time, measured in seconds, from the initial prompt submission to the final output generation or state-machine termination.

4.3 Execution Protocol

It is important to distinguish the overarching independent task attempts from the internal agentic looping of specific architectures. To prevent infinite hallucination loops within a single run of the Solver-Critic framework (the "ping-pong effect"), a hard safety brake was implemented within the LangGraph router. If the Critic rejected the Solver’s output 10 consecutive times, that specific execution graph was forcibly terminated, and the run was recorded as a failure for that trial.

5 Results

Table 1 consolidates the quantitative metrics across all 48 runs, revealing that architectural efficacy heavily depends on the task domain and structural difficulty.

Table 1: Comprehensive Evaluation Metrics across Architectures, Domains, and Difficulty Levels.

Domain	Difficulty	Architecture	Accuracy (%)	Avg. Tokens	Avg. Latency
ARC-AGI (Spatial)	Easy	Single LLM (Baseline)	25%	2,975	3.64s
		Solver-Critic	100%	57,617	95.86s
		Planner-Executor	75%	10,941	23.42s
	Hard	Single LLM (Baseline)	0%	2,157	1.61s
		Solver-Critic	75%	102,668	243.34s
		Planner-Executor	25%	9,212	23.14s
Scheduling (Constraint)	Easy	Single LLM (Baseline)	100%	127	0.84s
		Solver-Critic	100%	481	2.94s
		Planner-Executor	100%	2,764	14.6s
	Hard	Single LLM (Baseline)	50%	150	1.06s
		Solver-Critic	75%	676	3.27s
		Planner-Executor	100%	3,589	17.78s

5.1 Visual-Spatial Reasoning (ARC-AGI)

The ARC-AGI results highlight base models’ zero-shot limitations with spatial topologies and the extreme computational cost of text-based visual reasoning.

Table 2: Granular Task-by-Task Metrics for the ARC-AGI Easy Dataset.

Architecture	Task ID	Outcome	Tokens Consumed	Latency (s)	Loop Count
Single-Agent Baseline	Task 1	Correct	505	0.94	0
	Task 2	Incorrect	3,329	4.48	0
	Task 3	Incorrect	3,611	4.88	0
	Task 4	Incorrect	4,457	4.26	0
Solver-Critic	Task 1	Correct	1,918	4.20	1
	Task 2	Correct	22,337	20.10	2
	Task 3	Correct	166,953	340.84	9
	Task 4	Correct	39,260	36.40	3
Planner-Executor	Task 1	Correct	4,175	16.12	0
	Task 2	Correct	13,067	25.47	0
	Task 3	Correct	14,056	32.25	0
	Task 4	Incorrect	12,465	19.84	0

On Easy tasks (Table 2), the Baseline failed 75% of attempts. The Solver-Critic achieved 100% accuracy but exhibited massive token volatility; Task 3 consumed 166,953 tokens and took nearly six minutes. The Planner-Executor offered predictable token overhead ($\sim 12,000$) but failed unrecoverably on Task 4 due to an initial planning hallucination.

Table 3: Granular Task-by-Task Metrics for the ARC-AGI Hard Dataset.

Architecture	Task ID	Outcome	Tokens Consumed	Latency (s)	Loop Count
Single-Agent Baseline	Task 1	Incorrect	1,377	1.19	0
	Task 2	Incorrect	1,857	2.00	0
	Task 3	Incorrect	2,087	1.52	0
	Task 4	Incorrect	3,306	1.75	0
Solver-Critic	Task 1	Incorrect	115,468	267.47	10
	Task 2	Correct	87,231	245.58	8
	Task 3	Correct	143,765	334.03	10
	Task 4	Correct	64,208	126.29	4
Planner-Executor	Task 1	Incorrect	10,145	19.61	0
	Task 2	Incorrect	7,125	20.92	0
	Task 3	Correct	7,155	19.66	0
	Task 4	Incorrect	12,421	32.36	0

In the Hard dataset (Table 3), performance plummeted. The Baseline collapsed to 0%, and the Planner-Executor dropped to 25%, proving upfront planning fails on complex spatial scaling without visual state-checking. The Solver-Critic maintained 75% accuracy but pushed context limits. Task 1 hit the maximum iteration limit and failed after burning over 115,000 tokens, demonstrating a "reflection ceiling" where base models experience attention dilution rather than correcting geometric errors.

5.2 Constraint-Based Planning (Meeting Scheduling)

The constraint-based scheduling evaluation inverted the ARC-AGI paradigm, demonstrating base models' native competence with semantic rule execution.

On the Easy dataset (Table 4), all architectures achieved 100% accuracy. The Baseline executed flawlessly in under one second using only 126 tokens on average. Deploying multi-agent scaffolds here caused severe operational bloat, with the Planner-Executor burning nearly $20\times$ the resources for no additional gain.

Table 4: Granular Task-by-Task Metrics for the Meeting Scheduling Easy Dataset.

Architecture	Task ID	Outcome	Tokens Consumed	Latency (s)	Loop Count
Single-Agent Baseline	Task 1	Correct	127	0.85	0
	Task 2	Correct	133	1.02	0
	Task 3	Correct	131	0.66	0
	Task 4	Correct	115	0.82	0
Solver-Critic	Task 1	Correct	374	2.87	1
	Task 2	Correct	386	3.59	1
	Task 3	Correct	382	1.18	1
	Task 4	Correct	780	4.11	1
Planner-Executor	Task 1	Correct	2,544	13.76	0
	Task 2	Correct	3,426	17.51	0
	Task 3	Correct	2,819	14.94	0
	Task 4	Correct	2,268	12.19	0

On the Hard dataset (Table 5), multi-variable rules and timezone constraints reduced the Baseline’s accuracy to 50%. The Planner-Executor dominated, recovering a 100% success rate by systematically planning UTC conversions and using its execution trace to prevent cascading arithmetic errors. Conversely, the Solver-Critic failed Task 2 after a single loop; its text-based critique failed to detect the Solver’s underlying mathematical error, effectively rubber-stamping the hallucination.

Table 5: Granular Task-by-Task Metrics for the Meeting Scheduling Hard Dataset.

Architecture	Task ID	Outcome	Tokens Consumed	Latency (s)	Loop Count
Single-Agent Baseline	Task 1	Incorrect	149	0.87	0
	Task 2	Correct	145	0.92	0
	Task 3	Correct	158	1.43	0
	Task 4	Incorrect	147	0.92	0
Solver-Critic	Task 1	Correct	418	1.95	1
	Task 2	Incorrect	410	1.60	1
	Task 3	Correct	436	4.23	1
	Task 4	Correct	1,439	5.30	2
Planner-Executor	Task 1	Correct	2,444	13.98	0
	Task 2	Correct	3,781	19.45	0
	Task 3	Correct	5,240	23.86	0
	Task 4	Correct	2,892	13.82	0

6 Analysis

Empirical data reveals no universally superior agentic architecture. Instead, scaffolding efficacy is strictly domain-dependent, dictated by the task’s cognitive requirements and the base model’s native context limitations.

6.1 The Reflection Ceiling and Token Volatility (ARC-AGI)

In the visual-spatial domain, iterative reflection exposes a *reflection ceiling*. While Solver-Critic achieved 100% accuracy on Easy ARC-AGI tasks, it exhibited extreme token volatility (e.g., Task 3 consumed 166,953 tokens across 9 loops). On the Hard dataset, this volatility caused a

total architectural breakdown. Systems repeatedly hit the iteration limit (`MAX_RETRIES = 10`), with Task 1 burning 115,468 tokens before failing. This proves increasing loop allowances yields diminishing returns: expanding contexts cause severe attention dilution, locking the model into hallucination loops rather than synthesizing geometric feedback.

Additionally, the Planner-Executor’s collapse to 25% accuracy on Hard tasks confirms up-front decomposition fails for fluid intelligence; text-based models cannot reliably plan multi-step spatial transformations without continuous visual state-checking.

6.2 Architectural Overkill and Algorithmic Stability (Meeting Scheduling)

Conversely, the scheduling domain proves that multi-agent loops can degrade efficiency when tasks align with a model’s native semantic strengths. On Easy Scheduling tasks, the Baseline achieved 100% accuracy using just 126 tokens per task. Here, the Planner-Executor exhibited severe *architectural overkill*, consuming $20\times$ the resources for the exact same result.

However, on Hard tasks with multi-variable timezone constraints, Baseline accuracy dropped to 50%. The Planner-Executor became the optimal scaffold, maintaining 100% accuracy by explicitly planning UTC conversions and using an execution scratchpad to prevent cascading arithmetic errors. The Hard dataset also exposed a fatal flaw in Solver-Critic for rigid mathematics: it failed Task 2 after a single loop by rubber-stamping an arithmetic hallucination. This confirms linguistic critiques cannot reliably debug sequential math errors.

6.3 Hypothesis Validation

The recorded metrics definitively validate the initial hypotheses:

- **H1 (Domain Superiority) - Validated:** Planner-Executor dominated textual constraint sorting via sequential decomposition, whereas Solver-Critic provided the only viable recovery mechanism for abstract spatial transformations.
- **H2 (Difficulty Scaling) - Validated:** The Baseline matched multi-agent accuracy on Easy semantic tasks with superior efficiency. Scaffolding overhead is only justified when complexity exceeds the model’s zero-shot planning horizon.
- **H3 (Cost-Benefit Tradeoff) - Validated:** Solver-Critic consistently incurred the highest token and latency overhead. For structured constraints, the Planner-Executor is a vastly superior, cost-efficient compromise.

7 Conclusion

This research demonstrates that the efficacy of multi-agent scaffolding for Large Language Models is not universal, but strictly bounded by a task’s cognitive domain and the base model’s context limitations. Evaluating visual-spatial reasoning (ARC-AGI) against constraint satisfaction (Meeting Scheduling) establishes that architectures must dynamically match the problem’s underlying topology.

For text-based, sequential constraints, the **Planner-Executor** framework proved optimal. By externalizing arithmetic into a structured context buffer, it entirely mitigated zero-shot mathematical failures, achieving 100% accuracy on complex scheduling tasks. Conversely, applying iterative reflection to this algorithmic domain was structurally flawed, as linguistic Critic nodes failed to detect precise numeric errors, effectively rubber-stamping mathematical hallucinations.

In abstract spatial reasoning, however, baseline limitations remain profound. While the **Solver-Critic** architecture successfully recovered baseline failures on rudimentary visual tasks, it revealed an inherent *reflection ceiling*. Subjecting text-serialized 2D grids to deep iterative

critique induces extreme token volatility and attention collapse. The inability to reliably solve high-complexity spatial transformations without triggering maximum loop thresholds proves that text-based scaffolds cannot efficiently simulate fluid geometric intelligence.

Ultimately, while agentic workflows artificially extend a base model’s planning horizon, they face strict diminishing returns. Deploying heavy reflection loops on tasks where models possess native semantic competence causes severe architectural overkill, whereas deploying sequential planners on dynamic spatial tasks ensures unrecoverable geometric failures.

References

- [1] Chollet, F. (2019). *On the Measure of Intelligence*. arXiv preprint arXiv:1911.01547.
- [2] Moskvichev, A., Odouard, V. V., & Mitchell, M. (2023). *The Concept of “Concept” in ARC: Large Language Models Are Not Strong Abstract Reasoners*. arXiv preprint arXiv:2305.15821.
- [3] Greenblatt, R. (2024). *Getting 50% (SoTA) on ARC-AGI with GPT-4o*. Published via Substack / LessWrong.
- [4] ARC Prize Foundation. (2024). *OpenAI o3 Breakthrough High Score on ARC-AGI-Pub*. Retrieved from arcprize.org/blog/oai-o3-pub-breakthrough.
- [5] Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., & Yao, S. (2023). *Reflexion: Language Agents with Verbal Reinforcement Learning*. Advances in Neural Information Processing Systems (NeurIPS).
- [6] Wang, L., Xu, W., Lan, Y., Hu, Z., Lan, Y., Lee, R. K.-W., & Lim, E.-P. (2023). *Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning by Large Language Models*. Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL).